

SESSION 4: SELECT STATEMENT BASICS

1. Create a table named MusicPlaylist with columns: id, song_name, artist, genre, and duration. Insert at least 5 records representing songs from your favorite Spotify playlist, then write a SELECT statement to retrieve all columns for all songs.

```
mysql> CREATE TABLE MusicPlaylist (  
-> id INT PRIMARY KEY,  
-> song_name VARCHAR(100) NOT NULL,  
-> artist VARCHAR(100) NOT NULL,  
-> genre VARCHAR(100) NOT NULL,  
-> duration TIME  
-> );  
Query OK, 0 rows affected (16.358 sec)  
  
mysql> INSERT INTO MusicPlaylist  
-> (id, song_name, artist, genre, duration)  
-> VALUES  
-> (1, 'Shape of You', 'Ed Sheeran', 'Pop', '00:03:53'),  
-> (2, 'Perfect', 'Ed Sheeran', 'Romantic', '00:04:23'),  
-> (3, 'Blinding Lights', 'The Weeknd', 'Pop', '00:03:20'),  
-> (4, 'Believer', 'Imagine Dragons', 'Rock', '00:03:24'),  
-> (5, 'Havana', 'Camila Cabello', 'Pop', '00:03:37');  
Query OK, 5 rows affected (3.543 sec)  
Records: 5 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM MusicPlaylist;
```

id	song_name	artist	genre	duration
1	Shape of You	Ed Sheeran	Pop	00:03:53
2	Perfect	Ed Sheeran	Romantic	00:04:23
3	Blinding Lights	The Weeknd	Pop	00:03:20
4	Believer	Imagine Dragons	Rock	00:03:24
5	Havana	Camila Cabello	Pop	00:03:37

2. Write a SQL query to display only the song_name and artist columns from the MusicPlaylist table, showing just the first 3 records using the LIMIT keyword.

```
mysql> SELECT song_name, artist  
-> FROM MusicPlaylist  
-> LIMIT 3;
```

song_name	artist
Shape of You	Ed Sheeran
Perfect	Ed Sheeran
Blinding Lights	The Weeknd

3 rows in set (0.025 sec)

3. Suppose you have a table named FoodOrders with columns: id, restaurant, food_item, and order_date. Write a SQL query to list all unique restaurant names where you have placed orders, using the DISTINCT keyword.

```
mysql> CREATE TABLE FoodOrders (  
-> id INT PRIMARY KEY,  
-> restaurant VARCHAR(100),  
-> food_item VARCHAR(100),  
-> order_date DATE  
-> );  
Query OK, 0 rows affected (3.251 sec)  
  
mysql> INSERT INTO FoodOrders (id, restaurant, food_item, order_date)  
-> VALUES  
-> (1, 'Dominos', 'Pizza', '2026-10-01'),  
-> (2, 'McDonalds', 'Burger', '2026-10-02'),  
-> (3, 'Dominos', 'Pasta', '2026-10-03'),  
-> (4, 'KFC', 'Chicken Burger', '2026-10-04'),  
-> (5, 'McDonalds', 'French Fries', '2026-10-05');  
Query OK, 5 rows affected (0.652 sec)  
Records: 5 Duplicates: 0 Warnings: 0  
  
mysql> Select * from FoodOrders;  


| id | restaurant | food_item      | order_date |
|----|------------|----------------|------------|
| 1  | Dominos    | Pizza          | 2026-10-01 |
| 2  | McDonalds  | Burger         | 2026-10-02 |
| 3  | Dominos    | Pasta          | 2026-10-03 |
| 4  | KFC        | Chicken Burger | 2026-10-04 |
| 5  | McDonalds  | French Fries   | 2026-10-05 |

  
5 rows in set (0.423 sec)
```

```
mysql> SELECT DISTINCT restaurant  
-> FROM FoodOrders;  


| restaurant |
|------------|
| Dominos    |
| McDonalds  |
| KFC        |

  
3 rows in set (0.081 sec)
```

4. Write a SQL query on the FoodOrders table to select food_item as 'Dish' and order_date as 'Date Ordered', displaying only these two columns with the column aliases in the output.

```
mysql> SELECT food_item AS 'Dish',  
-> order_date AS 'Date Ordered'  
-> from FoodOrders;
```

Dish	Date Ordered
Pizza	2026-10-01
Burger	2026-10-02
Pasta	2026-10-03
Chicken Burger	2026-10-04
French Fries	2026-10-05

```
5 rows in set (0.033 sec)
```

5.You tried running this query: `SELECT DISTINCT food_item, restaurant FROM FoodOrders LIMIT 2,` but it returns an error or doesn't work as expected. Identify and fix the mistake in the query.

Hint: Check the correct placement and usage of the LIMIT keyword in SQL syntax.

```
mysql> SELECT DISTINCT food_item, restaurant
-> FROM FoodOrders
-> LIMIT 2;
```

food_item	restaurant
Pizza	Dominos
Burger	McDonalds

```
2 rows in set (0.112 sec)
```